

A Middleware Security Framework for Prompt Injection-Resilient LLM Agents

Project report

Cihan Ilhan

<https://github.com/IlhanCihan/LLMToolSecurityMiddleware>

1. Introduction

Large Language Models (LLMs) are now used as agents that can think through tasks and take actions by calling external tools such as search engines, databases, file systems, code executors, and workflow APIs. Research on tool-using agents (for example ReAct) shows that mixing natural-language reasoning with real actions can improve results and make the steps easier to follow [1]. At the same time, retrieval-augmented generation (RAG) is widely used to ground LLM outputs in external documents and organizational knowledge bases.

But when agents read untrusted text (web pages, PDFs, emails, tickets, logs, or tool outputs) in the same context window as trusted instructions, they can be tricked by prompt injection and instruction hijacking. Malicious content can look like “instructions” instead of “data”, which may lead to unsafe tool calls or leaking sensitive information [2]. Researchers also argue that prompt injection is different from classic injection bugs, so systems should be built to limit damage even if the model is manipulated.

The aim of this project is to design, implement, and evaluate a security middleware layer that sits between an LLM agent and its tools. Instead of relying on the LLM to decide what is safe, the middleware enforces capability-based permissions and policy checks at runtime. The project is guided by the following research questions:

How can we limit an agent to only the tools and permissions it needs, even during a prompt injection attempt?

How can we keep a clear boundary between trusted instructions and untrusted retrieved text (for example, web pages or PDFs)?

What is the trade-off between security and usefulness when we enforce stricter policies (false blocks, missed attacks, and latency)?

Can we produce audit logs that explain, in a human-readable way, why each tool call was allowed, denied, or sandboxed?

2. General project description

This project proposes a security middleware framework for tool-using LLM agents. The middleware sits between the agent and each tool call and decides, based on clear rules, whether

to allow, deny, or sandbox an action. The goal is to make the permission boundary explicit: the model can suggest actions, but the runtime enforces what the system is allowed to do.

The current deliverable includes a Python prototype that demonstrates: (i) trust and provenance (taint) tracking over tool arguments, (ii) simple prompt-injection heuristics, (iii) a policy engine with default-deny and explicit allow/sandbox rules, (iv) JSONL audit logging, and (v) a small evaluation harness with safe vs. adversarial scenarios.

3. Background and concepts

Tool-using agents: An “agent” is an LLM-driven loop that selects tools, sends arguments, reads outputs, and repeats. Many common patterns (for example ReAct and Toolformer) interleave reasoning steps with actions, so the model can consult external systems and correct itself.

Retrieval-augmented generation (RAG): RAG systems retrieve documents (often from a vector database) and place them into the model’s context to reduce hallucinations and keep answers grounded. In agent settings, retrieved text can also include adversarial instructions that try to redirect the agent.

Prompt injection and the “confused deputy” problem: Prompt injection is when an attacker writes input text so the LLM follows the attacker’s goals. If the LLM has access to privileged tools, it can become a “confused deputy” and make high-privilege requests on behalf of the attacker.

Capability-based security: Instead of giving broad tool access, the system assigns limited capabilities (for example read-only search or restricted file writes) and enforces them for each tool call. Capabilities can also restrict arguments (types, maximum lengths, allowed destinations), which reduces damage if the agent is tricked.

Provenance and taint tracking: The middleware labels inputs by trust level (system, developer, user, retrieved, tool output) and carries those labels into tool arguments. This enables policies such as: “deny writes when arguments are mostly derived from untrusted retrieved text”.

Auditability: Each enforcement decision should produce a structured log that records (i) the signals used, (ii) which policy rules matched, (iii) trust/provenance summaries, and (iv) the final action. Auditable traces help with debugging, compliance, and incident response.

4. State of things: current approaches and solutions

Defenses in many deployed LLM applications are still prompt-based (for example, stronger system prompts) or rely on the model to detect malicious instructions. In practice, these approaches are fragile because attackers can rephrase instructions, hide payloads in long documents, or exploit the model’s tendency to follow the most “important-looking” text.

Industry guidance (for example the OWASP Top 10 for LLM Applications) recommends defense in depth: validate inputs and outputs, design tools with least privilege, and monitor what the agent does. Prompt injection is listed as a major risk category.

Framework-level controls exist but are not always used as strict enforcement. For example, agent frameworks can intercept tool calls for logging and validation. However, many systems still treat tool invocation as an application detail instead of a security boundary.

Research has proposed runtime enforcement mechanisms for agent safety and reliability, including policy-driven execution and configurable guardrails. However, there is still no widely accepted benchmark for testing prompt injection defenses in realistic tool-using workflows, so it is hard to compare methods.

5. Problems, issues, and opportunity

The main gap is the lack of a reliable separation between trusted instructions and untrusted content inside the model’s context window. When retrieved documents and tool outputs influence the agent’s action choices, the model becomes a single point of failure for security decisions.

- Existing mitigations often lack:
- Hard runtime enforcement
- Fine-grained limits on tool arguments
- Provenance-aware decision-making
- Clear explanations for why an action was allowed or blocked. As agents move into real automation settings, these gaps can lead to risks such as data leakage, destructive actions, and unauthorized external calls.

The opportunity for this project is to treat tool calls as the privilege boundary and enforce rules outside the model. Even if prompt injection cannot be fully prevented, the system can reduce impact by constraining capabilities, sandboxing sensitive actions, and producing useful security logs.

6. Project overview: architecture and objectives

The proposed system follows a “security middleware” pattern: every tool call request passes through deterministic checks before execution. The current Python prototype implements the following pipeline:

- 1) Collect provenance (trust labels) from tool arguments and context (retrieved text, tool outputs).
- 2) Run prompt-injection detectors over tool arguments and untrusted text (regex/heuristic signals).
- 3) Compute security signals: injection score, untrusted ratio, declared intent, and available capabilities.
- 4) Validate tool arguments against capability constraints (type checks, max lengths, required fields).
- 5) Evaluate policy rules (default-deny; allow or sandbox only if explicit rules match).

- 6) Emit an audit log entry containing signals, matched rules, taint summary, and final decision.
- 7) Execute the tool directly (ALLOW) or via a sandbox function (SANDBOX); otherwise block (DENY).

In code terms, the main components are: (a) a ToolRegistry that maps tool names to callable functions (and optional sandbox variants), (b) PromptInjectionHeuristics that produces an injection score and evidence, (c) taint/provenance utilities that summarize trust origins, (d) a PolicyEngine that loads rules and capabilities from JSON and produces decisions, (e) a SecurityLogger that writes JSONL audit records, and (f) an Evaluator that runs scenarios and reports security/utility metrics.

Objectives for the full project (beyond the starter prototype) are: (O1) formalize the threat model for prompt injection in tool-using agents, (O2) implement a capability and policy model for common tool categories, (O3) improve provenance/taint tracking across multi-step agent workflows, (O4) build a benchmark with realistic workflows and adversarial injections, and (O5) evaluate security improvements vs. utility loss (task completion, false blocks, and latency).

7. Planning for the next semesters

The project will follow an iterative build-and-evaluate cycle, aligned with a two-year master project timeline:

Year 1 / Semester 2: stabilize the middleware prototype (fix parsing and formatting issues, finish the intent classifier stub), integrate with a small agent + RAG setup, and add richer policies (URL allowlists, path constraints, and rate limits).

8. References

- [1] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629.
- [2] Liu, Y., Deng, G., Li, Y., Wang, K., Wang, Z., Wang, X., Zhang, T., Liu, Y., Wang, H., Zheng, Y., & Liu, Y. (2023). Prompt Injection attack against LLM-integrated Applications. arXiv:2306.05499.